

Machine Learning Project

Milestones 1 & 2 — Final Report

Team ID: SC_21

Name	Student ID
Nadine Hassan El-Sayed Ahmed	2023170655
Yasmine Mohamed Obaid Mohamed	2023170698
Mostafa Mohamed Abd El-Hamid El-Kafrawy	2023170609
Mohab Ahmed Abd El-Hamid Mohamed	2023170643
Nour Dia El-Din Ahmed Talaat Amin	2023170671

April – May 2026

1. Project Overview

This report covers both milestones of the Machine Learning project for Team SC_21. The dataset is the Open University Learning Analytics Dataset (OULAD), which captures student activity and demographics across several course modules. In **Milestone 1** we treated student score prediction as a regression problem. In **Milestone 2** we reframed it as a four-class classification problem — predicting whether a student's assessment result falls into **Fail**, **Good**, **Very Good**, or **Excellent**.

Both tasks share the same seven source CSV files and the same core preprocessing pipeline. Where they diverge — encoding strategy, feature set, target variable — we explain the reasoning explicitly.

2. Dataset Overview

We worked with seven CSV files:

- **studentAssessment** — 163,912 submission records linking students to assessment scores.
- **assessments** — 206 assessments with type, due date, and weight.
- **studentInfo** — 32,593 student demographic records (gender, region, IMD band, education level, disability).
- **studentRegistration** — 32,593 registration and unregistration records.
- **studentVle** — 10,655,280 click-level VLE activity logs.
- **vle** — 6,364 VLE module resources with activity type.
- **courses** — 22 module-presentation combinations with total module length in days.

After merging and cleaning: the regression dataset has 163,752 rows (score-bearing submissions only); the classification dataset has 163,912 rows (all submissions, target derived from score bands). Both datasets have over 65 features before engineering.

3. Data Preprocessing

3.1 Merging the Datasets

We first aggregated the VLE click logs by student and module. For each student-module pair we computed five summary statistics from studentVle (joined with vle for activity type): **total_clicks**, **avg_clicks**, **interaction_days**, **total_interactions**, and **unique_activity_types**. We then joined studentAssessment with assessments, studentInfo, courses, studentRegistration, and the aggregated VLE data, using student ID, module code, and presentation code as keys.

The **final_result** column in studentInfo was dropped immediately — it encodes whether the student passed the module overall, which would be data leakage for score prediction.

3.2 Handling Missing Values

Each column with missing data was handled based on what the missingness actually means:

- **score (regression target)** — Rows deleted — no point keeping a row with no prediction target.
- **date**, **date_registration** — Filled with the column median. These date columns are right-skewed, so the median is more representative than the mean.

- **date_unregistration** — Converted to a binary flag: **has_unregistered** = 1 if a date existed, 0 otherwise. The column itself was then dropped.
- **imd_band (regression)** — Filled with the mode (most common band).
- **imd_band (classification)** — Filled with 'Unknown' — a distinct category — so one-hot encoding could capture the missingness as its own signal.
- **VLE columns** — Filled with 0. Missing VLE records mean no recorded activity.

3.3 Type Conversions & Cleaning

Several numeric columns were stored as text due to '?' placeholder characters. We replaced those with NaN and converted the columns to numeric. After cleaning, zero nulls remained in the final feature matrix.

3.4 Encoding Categorical Variables

For **regression**, we used ordinal encoding: ordered categories (imd_band, highest_education, age_band) received numbers that match their natural ranking; unordered ones (code_module, assessment_type, gender, region, disability) received arbitrary numeric labels.

For **classification**, we used one-hot encoding across all categorical columns. This expanded the feature space to 65 columns before engineering and is more appropriate for tree-based classifiers that benefit from explicit binary splits per category level.

3.5 Removing Duplicates

After merging, we checked for and removed duplicate rows to avoid training on the same data point more than once.

4. Data Analysis & Correlation

4.1 Score Distribution (Regression)

The target variable (score) has a mean of 75.8 and std of 18.8. Most students score between 70 and 100, with a spike at exactly 100. A small tail of near-zero scores exists — students who submitted but barely engaged. Raw total_clicks is heavily right-skewed; we applied a log transform to compress the range.

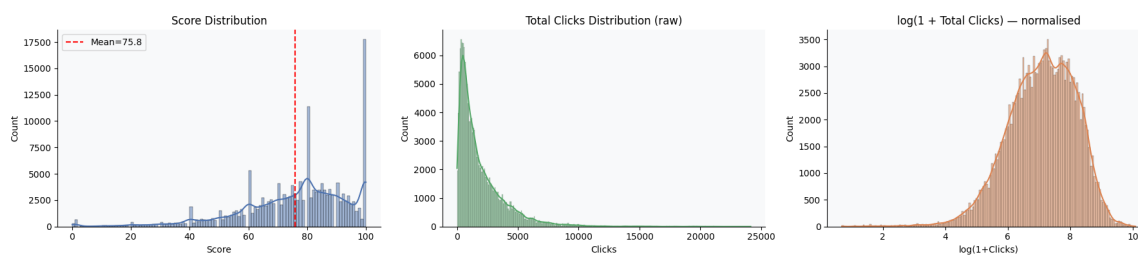


Figure 1: Score distribution (left), raw total clicks (centre), and log-normalised total clicks (right).

4.2 Correlation Matrix

Correlations across all numeric features revealed: engagement breadth and depth go together (interaction_days and total_clicks are positively correlated); avg_clicks_per_day and total_clicks are highly correlated with each other; and no single feature dominates score prediction — all correlations with score are moderate at best. Gender, region, and disability showed near-zero correlation with score.

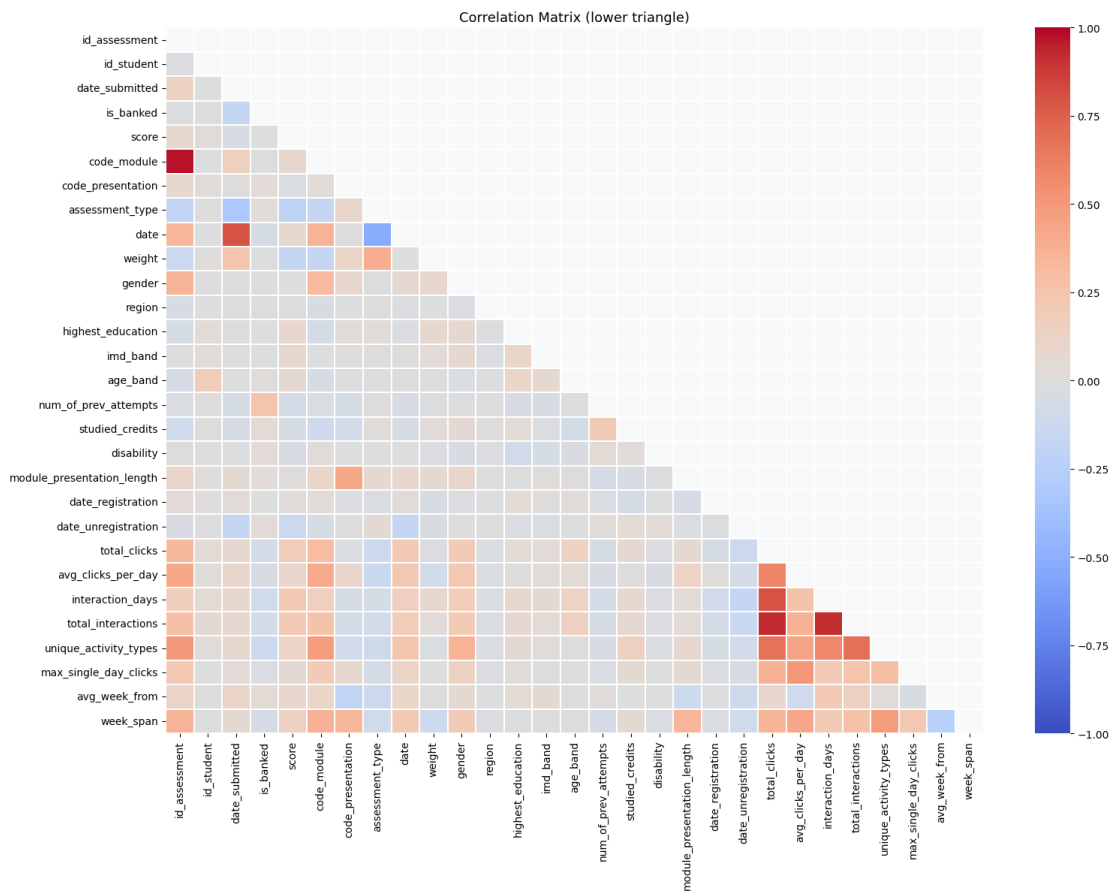


Figure 2: Correlation matrix (lower triangle). Darker red = stronger positive; darker blue = stronger negative.

4.3 IMD Band vs. Score

Plotting IMD band (deprivation index) against score showed that score distributions are broadly similar across all bands, with nearly identical medians. This is consistent with the near-zero correlation we observed and explains why `imd_band` ends up as a lower-ranked feature.

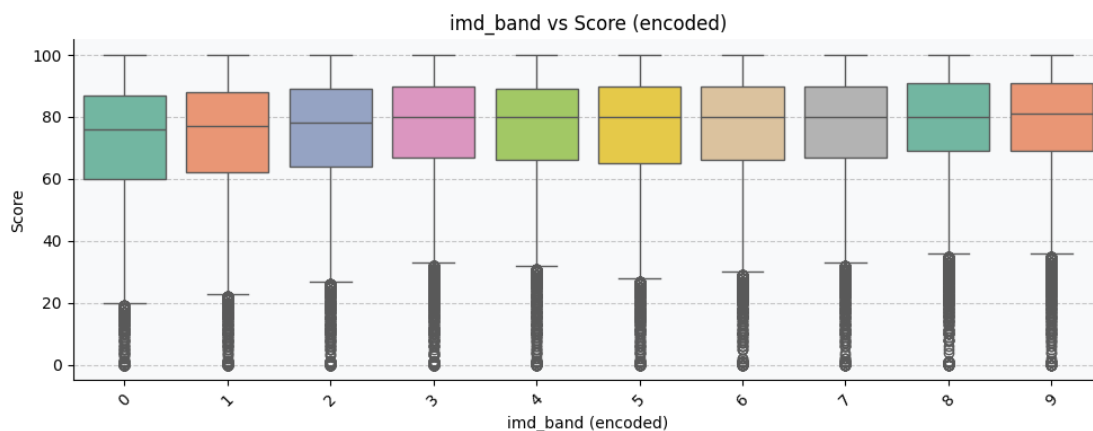


Figure 3: Score distribution across encoded IMD bands. Patterns are largely consistent across bands.

4.4 Class Distribution (Classification)

The four assessment classes are: Very Good (57,623 — 35%), Excellent (41,353 — 25%), Good (38,636 — 24%), Fail (26,300 — 16%). The dataset is moderately imbalanced — Fail is under-represented. We addressed this in Logistic Regression and Random Forest by setting `class_weight='balanced'`.

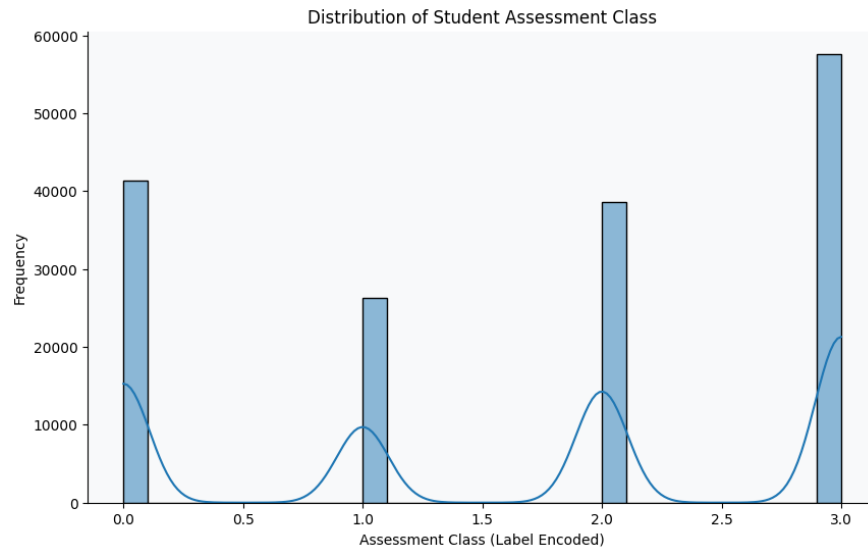


Figure 4: Distribution of assessment class labels (label-encoded: 0=Excellent, 1=Fail, 2=Good, 3=Very Good).

4.5 Top Features Correlated with Assessment Class

Among all original features, `assessment_type_TMA` ($r=0.128$) and `assessment_type_CMA` ($r=-0.121$) show the highest absolute correlations with class label, followed by `code_module_FFF` and `unique_activity_types`. Gender and age_band appear in the lower half of the ranking.

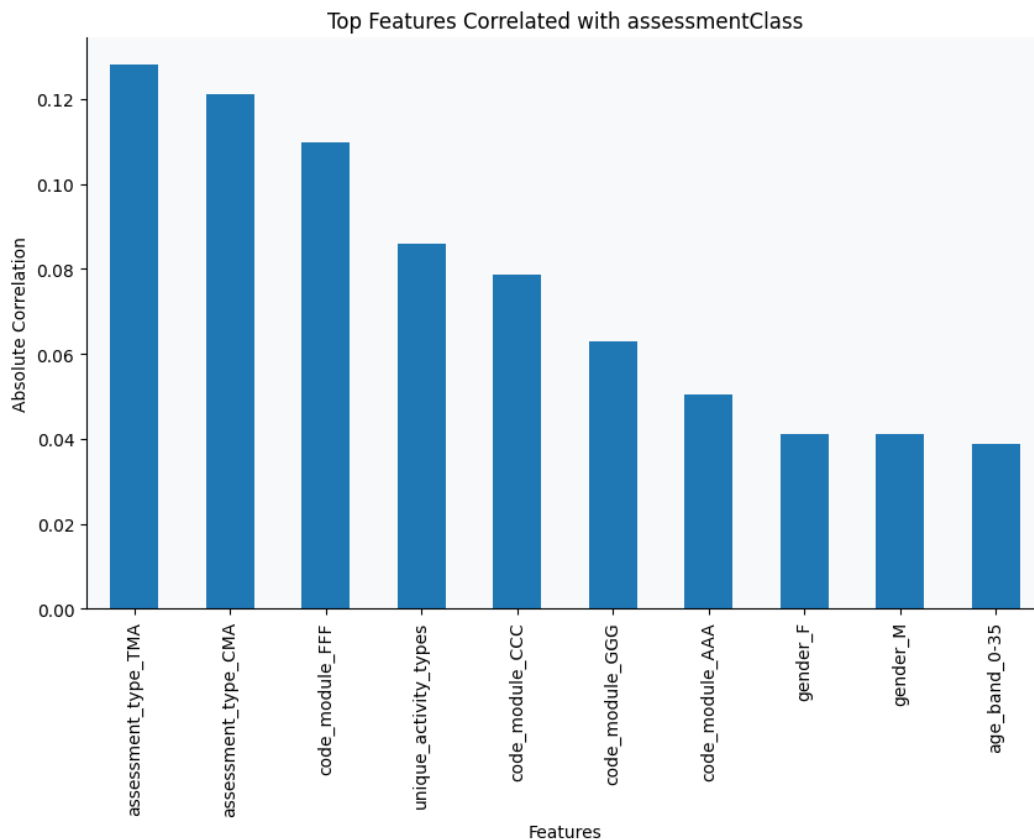


Figure 5: Top 10 original features correlated with assessment class (absolute values).

5. Feature Engineering & Selection

5.1 New Features — Shared Across Both Tasks

The same feature engineering code was applied to both regression and classification. We created three groups of new features:

Submission timing

- **submission_offset** — Days between submission and the assessment due date. Negative = early, positive = late.
- **submitted_late** — Binary: 1 if submission_offset > 0.
- **submitted_early** — Binary: 1 if submission_offset < -3.
- **submission_timing_ratio** — submission_offset / (module_presentation_length + 1) — normalised across modules of different lengths.

Engagement

- **click_per_day** — total_clicks / (interaction_days + 1) — click intensity on active days.
- **avg_clicks_per_interaction** — total_clicks / (total_interactions + 1) — quality of engagement per resource.
- **activity_diversity_ratio** — unique_activity_types / (total_interactions + 1).
- **engagement_ratio** — interaction_days / module_length, clipped at 1.0.
- **is_heavy_engager** — Binary: 1 if total_clicks ≥ 75th percentile (threshold = 2,695 clicks).

Student behaviour

- **registration_offset** — Equals date_registration — how far before the module start the student enrolled.
- **has_prior_attempts** — Binary: 1 if num_of_prev_attempts > 0.
- **high_credit_load** — Binary: 1 if studied_credits > median (60 credits).

Interaction flags

- **engaged_but_late** — Heavy engager who still submitted late — high effort, poor time management.
- **low_engagement_late** — Low engager who also submitted late — double risk signal.

After adding these features, we also dropped avg_clicks, total_interactions, and interaction_days since they are subsumed by the engineered versions.

5.2 Engineered Feature Correlations with Target

Figure 6 shows the absolute correlation of each engineered feature with the classification target. **submitted_late** ($|r| = 0.124$) and **low_engagement_late** ($|r| = 0.113$) lead by a wide margin — confirming that submission timing is the most informative new signal. Pure engagement volume (is_heavy_engager) correlates less than the timing-behaviour interaction features.

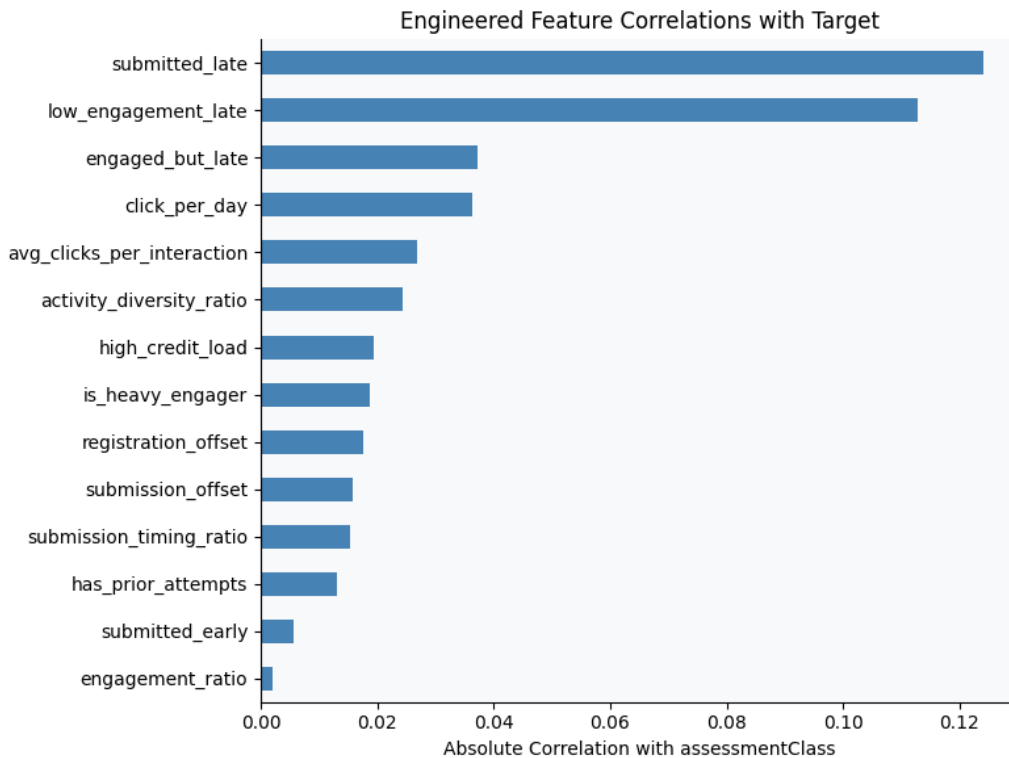


Figure 6: Absolute correlation of engineered features with assessment class. Submission timing dominates.

5.3 Feature Selection — Regression

The following columns were removed for regression:

- **id_assessment, id_student** — Pure ID columns with no predictive meaning.
- **total_clicks, avg_clicks_per_day, total_interactions, max_single_day_clicks** — Replaced by their log-transformed versions.
- **date_submitted, date, date_registration** — Replaced by submission_timing.
- **is_banked** — Near-zero correlation with score.
- **gender, region** — Near-zero correlation with score in both tasks.

The final regression feature set contains 18 curated features, including assessment_type, weight, interaction_days (pre-drop), submission_timing, log-transformed click features, and the new engineered signals.

5.4 Feature Selection — Classification

For classification, all 75 features (after one-hot encoding and engineering) were retained. Tree-based classifiers handle uninformative features without manual selection, and the one-hot expanded representation is appropriate for classifiers that split on individual feature values.

5.5 Feature Importance (Regression)

assessment_type (~16%) and **weight** (~13.5%) are the top two features in Gradient Boosting — a final exam and a short quiz are scored on completely different scales, and assessment weight directly affects how much a submission counts. Behavioral features follow: interaction_days, submission_timing, and log_total_interactions all appear in the top five. On the Ridge side, weight and weighted_engagement dominate, showing that linear models lean more heavily on the aggregated engagement signal.

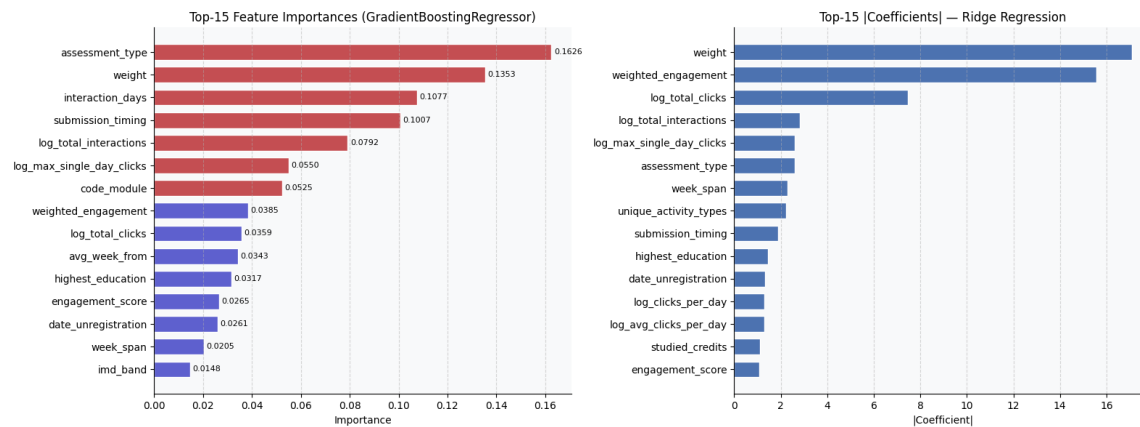


Figure 7: Top-15 feature importances from Gradient Boosting (left) and top-15 |coefficients| from Ridge (right).

6. Regression — Milestone 1

6.1 Models Trained

Five regression models were trained and compared:

- **Ridge Regression** — Linear model with L2 regularisation. Stable baseline for linear relationships.
- **Lasso Regression** — L1 regularisation — shrinks some coefficients to zero, acting as automatic feature selection.
- **Decision Tree** — Single tree splitting on feature thresholds. Interpretable but weaker than ensembles.
- **Random Forest** — Many independent trees averaged. Handles non-linearity and is more stable than a single tree.
- **Gradient Boosting** — Sequential trees, each correcting the errors of the previous. Our strongest model.

6.2 Train / Validation / Test Split

70% training, 15% validation (model selection), 15% test (final evaluation, used once). StandardScaler was fitted on the training set only; the same parameters were applied to validation and test sets to prevent information leakage.

6.3 Validation Results

Table 1 shows validation-set performance. Gradient Boosting ranked first on all three metrics. The two linear models scored considerably lower — a clear signal that the score-feature relationship is non-linear and ensemble methods are better suited.

Model	R ²	RMSE	MAE	Rank
Gradient Boosting	0.2800	15.9	12.1	1
Random Forest	0.2479	16.3	12.4	2
Decision Tree	0.2023	16.8	12.8	3
Ridge	0.1489	17.3	13.3	4
Lasso	0.1092	17.7	13.6	5

Table 1: Validation set performance for all five regression models.

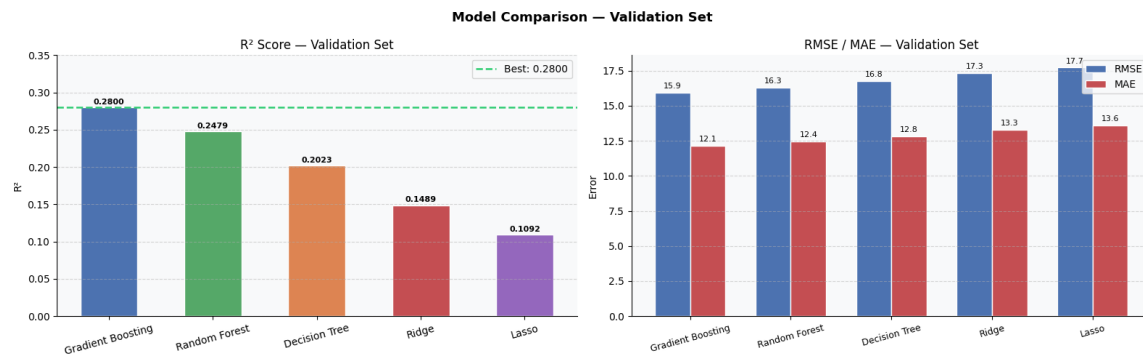


Figure 8: R^2 scores (left) and RMSE/MAE (right) on the validation set. Gradient Boosting leads both panels.

6.4 Actual vs. Predicted

In the scatter plots below, a perfect model would put all points on the red diagonal. Gradient Boosting clusters closest to that line. Ridge and Lasso compress predictions toward the middle of the range and miss the extremes. Decision Tree produces a blocky pattern — it maps many students to the same fixed leaf value.

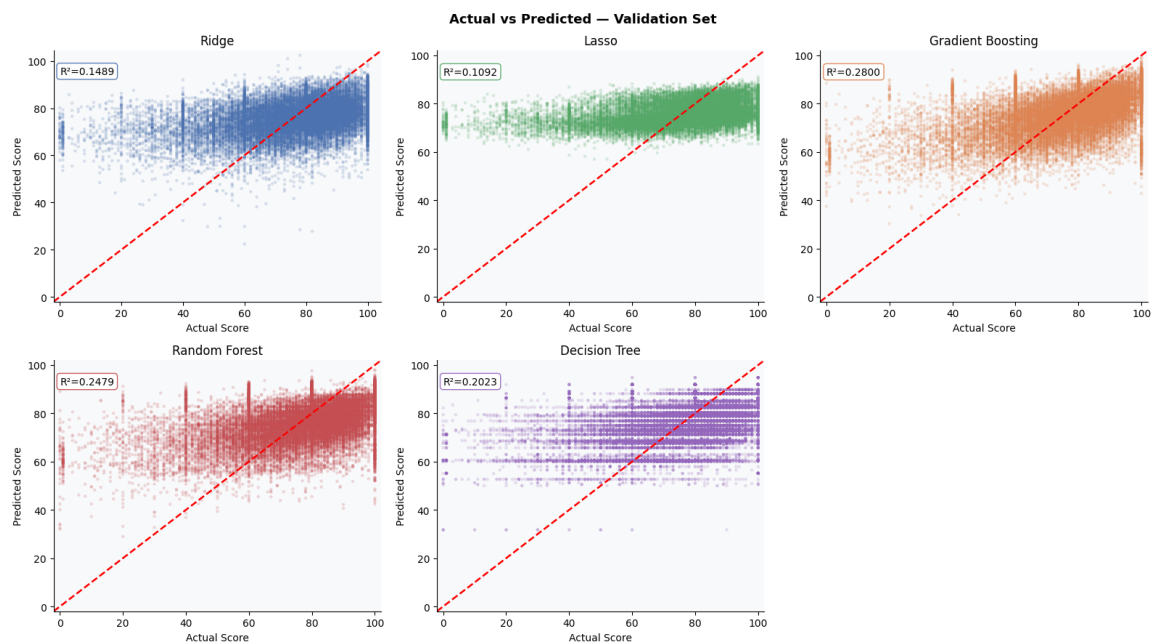


Figure 9: Actual vs. predicted score for all five models on the validation set.

6.5 Overfitting Check & Learning Curve

Figure 10 compares Train/Val/Test R^2 and shows the Gradient Boosting learning curve. All models have small train-test gaps, meaning regularisation worked. Gradient Boosting's learning curve shows the validation score converging toward the training score as more data is added — a healthy generalisation pattern.

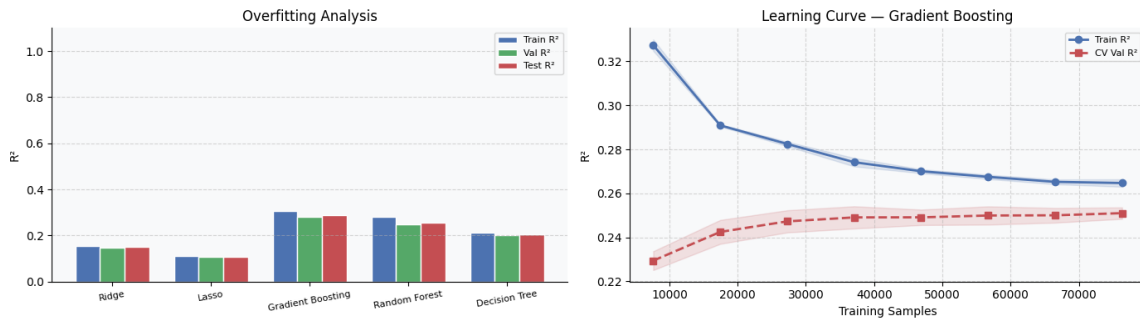


Figure 10: Overfitting analysis — Train/Val/Test R^2 for all models (left); Gradient Boosting learning curve (right).

6.6 Cross-Validation

5-fold cross-validation on the training data confirmed the ranking. Gradient Boosting scored highest across all five folds.

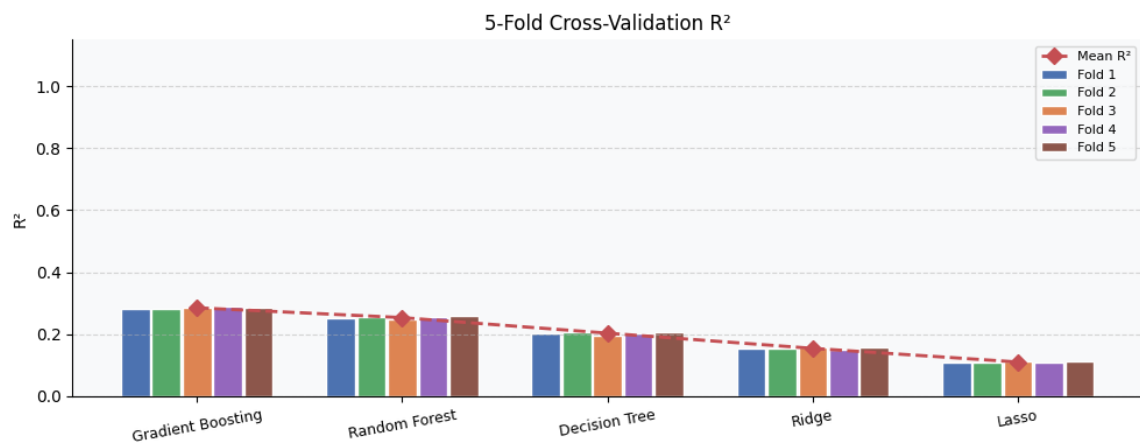


Figure 11: 5-fold cross-validation R^2 per model. Gradient Boosting leads consistently in all folds.

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean R^2	Std
Gradient Boosting	0.2838	0.2847	0.2895	0.2871	0.2854	0.2861	0.0020
Random Forest	0.2478	0.2490	0.2526	0.2534	0.2533	0.2512	0.0023
Decision Tree	0.2079	0.2071	0.2124	0.2134	0.2136	0.2109	0.0028
Ridge	0.1762	0.1711	0.1734	0.1710	0.1710	0.1725	0.0020
Lasso	0.1267	0.1238	0.1235	0.1237	0.1259	0.1247	0.0013

Table 2: 5-fold cross-validation results. Mean R^2 and standard deviation across folds.

6.7 Residual Analysis (Gradient Boosting)

The residual histogram is approximately bell-shaped and centred near zero (mean error = 0.15), meaning the model is not systematically biased in either direction. The residuals-vs-predicted plot shows a fan shape — errors grow larger for students predicted to score above 80. High scorers are harder to pin down: top performance has more random variance and fewer consistent behavioural predictors.

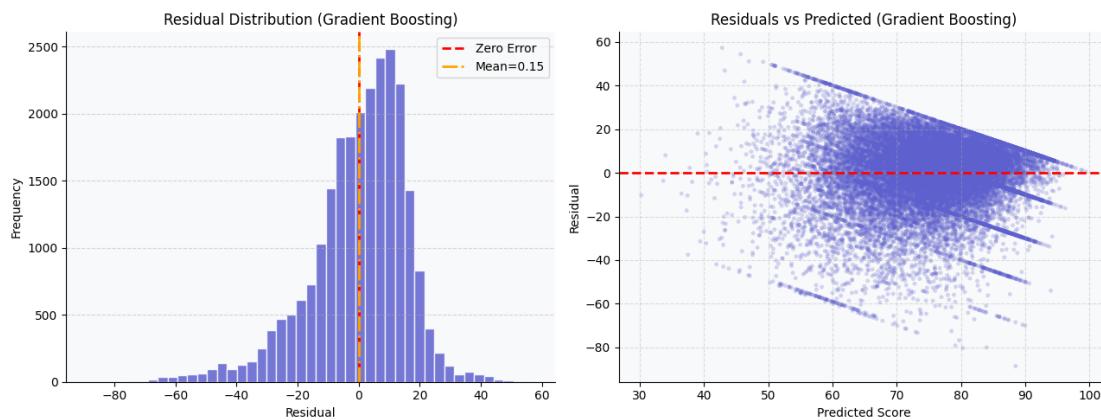


Figure 12: Gradient Boosting residual distribution (left) and residuals vs. predicted score (right).

6.8 Regression Conclusion

Gradient Boosting was the clear winner with $R^2 = 0.28$ and $RMSE = 15.9$. The model explains about 28% of the variance in assessment scores — meaningful, but leaving 72% unexplained. That gap is expected: how well a student actually understood the material, their motivation, and the specific difficulty of each question simply don't appear in any clickstream log.

Assessment structure (type and weight) drove prediction more than engagement volume. A final exam and a short quiz are designed differently, and that design difference affects scores more than how much a student clicked around on the platform.

7. Classification — Milestone 2

7.1 Problem Definition & Target Variable

In Milestone 2 we reframed prediction as a four-class classification task. The `assessmentClass` column encodes performance bands: **0 = Excellent**, **1 = Fail**, **2 = Good**, **3 = Very Good**. Class distribution: Very Good (35%), Excellent (25%), Good (24%), Fail (16%). The moderate imbalance was addressed with `class_weight='balanced'` in Logistic Regression and Random Forest.

7.2 Key Preprocessing Differences from Regression

The core pipeline is identical. The differences are:

- One-hot encoding for all categorical columns instead of ordinal encoding.
- `imd_band` missing values filled with 'Unknown' category rather than the mode.
- All 75 engineered features retained — tree classifiers handle irrelevant features natively.
- 80/20 stratified train/test split (no separate validation set); hyperparameters explored by sweeping and comparing test accuracy.

7.3 Models Trained

Four classifiers were trained and compared: Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting. SVM was excluded due to prohibitive training time on 130K+ samples.

7.4 Hyperparameter Tuning

For each model we swept two key hyperparameters and tracked test accuracy. Key findings: Logistic Regression accuracy is almost insensitive to `C` but responds to solver choice. Random Forest benefited most from increasing `max_depth` (5 → 10 → 20: +9pp accuracy) at the cost of a larger train-test gap. Gradient Boosting improved steadily with higher learning rate — `lr=0.2` with `n=200` was the best configuration but cost ~8.5 minutes to train.

Logistic Regression — C sweep (balanced class_weight)		Train Acc	Test Acc
	0.001	0.4117	0.4111
	1	0.4122	0.4098
	100	0.4123	0.4098
Solver sweep (C=1.0, no class_weight)	lbfgs	0.4420	0.4418
	newton-cg	0.4419	0.4420
	sag	0.4419	0.4417

Table 3: Logistic Regression hyperparameter sweep. Solver matters more than `C` in this range.

Random Forest	Hyperparameter	Value	Train Acc	Test Acc
n_estimators sweep (max_depth=10)	n_estimators	50	0.4761	0.4449
	n_estimators	100	0.4775	0.4459
	n_estimators	200	0.4775	0.4467
max_depth sweep (n=100)	max_depth	5	0.4233	0.4219

	max_depth	10	0.4775	0.4459
	max_depth	20	0.9061	0.5122 ★

Table 4: Random Forest sweep. max_depth=20 gives best test accuracy (0.5122) but causes significant overfitting (train acc = 0.91).

Gradient Boosting	learning_rate	n_estimators	Train Acc	Test Acc	Train Time
lr sweep (n=200)	0.05	200	0.4754	0.4737	512s
	0.10	200	0.4860	0.4796	501s
	0.20	200	0.5039	0.4833 ★	500s
n sweep (lr=0.2)	0.20	30	0.4686	0.4687	75s
	0.20	50	0.4759	0.4734	126s
	0.20	80	0.4832	0.4766	200s

Table 5: Gradient Boosting sweep. Higher learning_rate improved accuracy; more estimators had diminishing returns.

Decision Tree	Hyperparameter	Value	Train Acc	Test Acc
min_samples_split sweep (max_depth=10)	min_samples_split	2	0.4863	0.4562
	min_samples_split	5	0.4861	0.4562
	min_samples_split	10	0.4856	0.4561
min_samples_leaf sweep (max_depth=10)	min_samples_leaf	1	0.4863	0.4562
	min_samples_leaf	2	0.4859	0.4561
	min_samples_leaf	4	0.4852	0.4558

Table 6: Decision Tree sweep. Results are almost identical across all configurations — the model is at its capacity at max_depth=10.

7.5 Model Comparison

Figure 13 shows test accuracy, total training time, and inference time side by side. Random Forest achieves the best accuracy (51.2%) with a reasonable training time (42s). Gradient Boosting is competitive on accuracy (48.3%) but over 13× slower to train. Decision Tree is the fastest by far at under 4 seconds total.

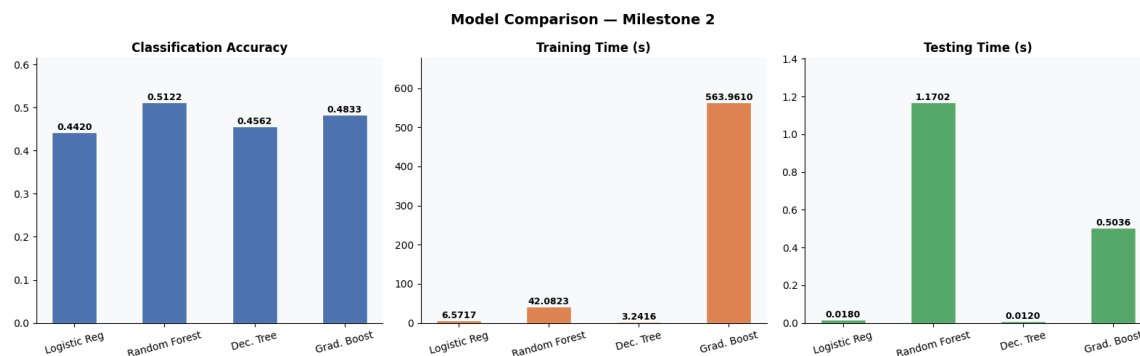


Figure 13: Classification accuracy (left), training time in seconds (centre), and inference time (right).

Model	Train Acc	Test Acc	Macro F1	Train Time	Test Time
Logistic Regression	0.4419	0.4420	0.4086	6.57s	0.0180s

Random Forest	0.9061	0.5122	0.5081	42.08s	1.1702s
Decision Tree	0.4863	0.4562	0.4336	3.24s	0.0120s
Gradient Boosting	0.5039	0.4833	0.4627	563.96s	0.5036s

Table 7: Full classification results. Random Forest is the recommended model — best test accuracy and Macro F1.

7.6 Evaluation — Per-Class Metrics

Table 8 summarises per-class precision, recall, and F1 for the two best models. Random Forest is the most balanced: it achieves 0.63 recall for Excellent and 0.55 for Fail. Logistic Regression concentrates predictions on Very Good (the majority class), leading to high recall for that class but very poor recall for Good (0.18). Good is the hardest class to predict across all models — its score band overlaps significantly with both Fail and Very Good in terms of behavioural signals.

Class	LR Prec.	LR Rec.	LR F1	RF Prec.	RF Rec.	RF F1
Excellent	0.51	0.49	0.50	0.58	0.63	0.61
Fail	0.44	0.35	0.39	0.48	0.55	0.51
Good	0.38	0.18	0.24	0.46	0.38	0.40
Very Good	0.42	0.63	0.51	0.51	0.50	0.52
Macro avg	0.44	0.41	0.41	0.51	0.52	0.51

Table 8: Per-class metrics for Logistic Regression and Random Forest (best configs).

7.7 Confusion Matrices

Figure 14 shows normalised confusion matrices for all four models. All models correctly favour the diagonal. Random Forest has the strongest diagonal values overall. The Good class (row 3) consistently has the most off-diagonal mass — students predicted as Very Good when they were actually Good is a common error across all classifiers.

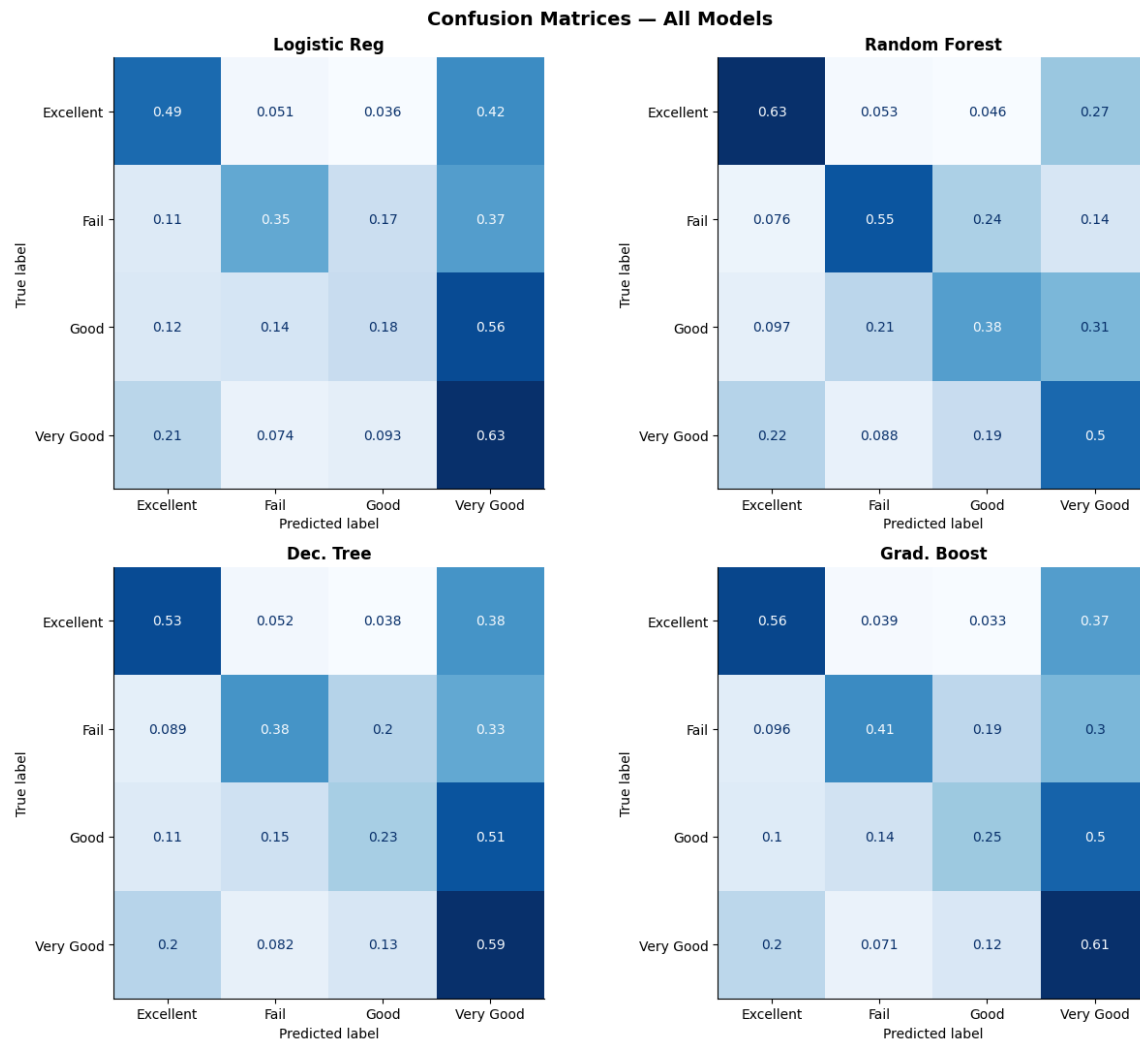


Figure 14: Normalised confusion matrices for all four classifiers. Darker = higher proportion.

7.8 Classification Conclusion

Random Forest (max_depth=20) is the recommended model: best test accuracy (51.2%) and best Macro F1 (0.508). Its large train-test gap (0.91 vs. 0.51) signals overfitting at depth 20, but it still generalises better than any other model. Gradient Boosting is a close second on F1 (0.463) but takes nearly 10 minutes to train.

A 51% accuracy on a four-class problem is a real result — chance is 25%. The Good class remains the consistent weak point across all models, likely because students in that band don't exhibit a sharply distinct behavioural pattern compared to their neighbours.

Submission timing turned out to be the most informative engineered signal — students who submit late are disproportionately likely to land in Fail or Good. We expected raw engagement volume (clicks) to dominate; instead, when and how they submit matters more than how much they click.

8. Final Conclusion

Across both milestones, the same pattern appears: **how an assessment is structured matters more than raw engagement volume**. Assessment type and weight dominated regression feature importances. Submission timing dominated classification feature correlations. Total clicks and active days matter, but they're secondary to structural and timing signals.

On the regression side, **Gradient Boosting** ($R^2 = 0.28$, RMSE = 15.9) was the clear winner. An R^2 of 0.28 is honest — most of what determines a specific numeric score isn't captured in platform logs. Tree-based models consistently beat linear models, confirming that the relationship is non-linear.

On the classification side, **Random Forest** (test accuracy = 51.2%, Macro F1 = 0.508) came out on top. The Good class is the hardest to classify across all models — its behavioural fingerprint overlaps too heavily with neighbouring classes. That's a genuine data limitation, not a modelling failure.

The project reinforces that education prediction is hard not because models fail, but because student performance has a large irreducible component — understanding, motivation, and circumstances that no clickstream can fully capture.

End of Report